NBA History Analyst Agent

A natural-language analytics agent over Basketball-Reference's season-level NBA, ABA and BAA data (Kaggle: `sumitrodatta/nba-aba-baa-stats`). A user asks a basketball question in plain English; the agent looks up the schema, writes SQL against the CSV-backed tables, checks the result, and answers in plain English with the numbers and the filters it used. It runs on OpenAI or Google Gemini models, in Google Colab.

Why this project: it is a small, fully measurable version of the "ask questions of our business data" problem. Swap these CSVs for corporate extracts and the same architecture, evaluation method, and failure modes apply. The dataset even contains the classic traps (duplicate rows from totals, missing-value markers, an off-by-one date convention) that corporate data has.

Start here: what to do next in Colab

You have the Kaggle dataset downloaded in Colab and API keys for OpenAI and Gemini. Do these in order. Each step has a "done when" check.

#	Do this	Done when
1	Open `notebooks/colab_runner.ipynb` in Colab. Add `OPENAI_API_KEY` and `GEMINI_API_KEY` under **Secrets** (key icon, left sidebar) and switch on notebook access for each. Run the clone/install cell.	`python src/list_models.py` prints model names for both providers
2	Find the folder with the Kaggle CSVs (Colab file browser). Put its path in `DATA_DIR` in the notebook, or set `NBA_DATA_DIR`.	`python src/inspect_csvs.py <folder> --files-only` shows `[ok]` for all 12 expected files
3	Run `python src/check_data.py <folder>`.	All sanity checks say `[ok]`, and the top scorers match Basketball-Reference
4	Run `python -m pytest tests -q`, then `python src/evaluate.py --data-dir <folder> --check-gold`. Compare 3–4 gold answers with Basketball-Reference.	Tests pass; gold answers look right
5	Ask the agent one question by hand with each provider (notebook step 4).	You get an answer plus the SQL it ran
6	`python src/evaluate.py --provider openai --split dev --limit 5 --judge`, check the cost in the provider dashboard, then run `--split dev` in full for each provider.	You have `results/eval_*.csv` for both providers
7	Read the failures, pick one fix (section 5), rerun on `--split dev`. Repeat. When you stop changing things, run `--split test` **once**.	A before/after table on dev, and one final score on test

#	Do this	Done when
8	Have the models build the app (section 6): `python src/build_app.py --provider gemini`, read the file, launch it.	A chat interface you can open and share
9	Gemma comes after this works.	

About SQL in Colab: you do not need a database. DuckDB is the SQL engine: it installs with one `pip install` (already in `requirements.txt`), runs inside the notebook, and queries your CSV files directly. Your data stays CSV.

If a file or column is missing, `check_data.py` says exactly which one. If your Kaggle copy is a newer version of the dataset and a file or column was renamed, edit `src/schema.py` and the views at the top of `src/db.py`.

1. Goals and non-goals

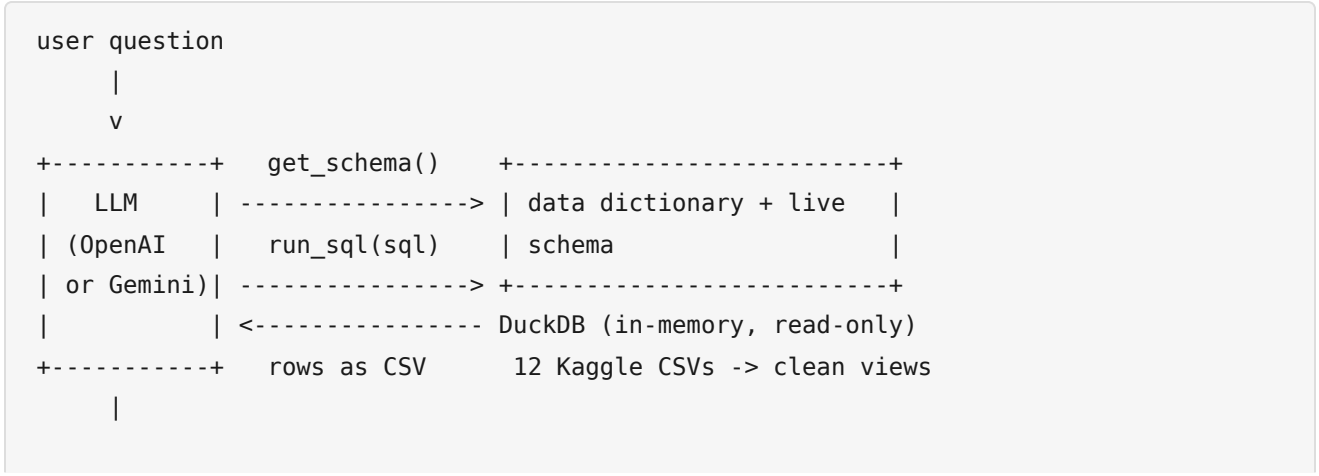
Goals

1. Answer factual and analytical basketball questions correctly from data, with no invented statistics.
2. Say "the data can't answer that" when it can't (game logs, playoff player stats, salaries, predictions).
3. Measure accuracy with a fixed test set, then improve it and show the before/after.
4. Compare OpenAI and Gemini models on accuracy, cost and speed.
5. Document what transfers to a business setting.

Non-goals: training an LLM from scratch (a Gemma comparison comes later), live scores, predictions.

Design principle: the model never *remembers* statistics. It *finds* them. Numbers come only from query results, which is what makes the answers checkable.

2. Architecture



```
      v
plain-English answer + filters used
```

Piece	File	Job
Data layer	`src/db.py`, `src/schema.py`	Loads the CSVs into DuckDB, defines the clean views, guards SQL
Data checks	`src/check_data.py`, `src/inspect_csvs.py`	Confirm the files and views work; look inside the CSVs
Agent	`src/agent.py`	System prompt, two tools, and the question-to-answer flow
LLM layer	`src/llm.py`	One tool-calling loop each for OpenAI (Responses API) and Gemini (`google-genai`), with retry on rate limits
Models	`src/list_models.py`	Lists the models your API keys can use
Eval	`src/evaluate.py`, `eval/questions.csv`	42 questions with gold SQL (28 dev, 14 held-out test); automatic + LLM-judge grading
UI	`src/app.py`, `src/build_app.py`, `docs/app_brief.md`	Plain Gradio chat, plus a script that has OpenAI or Gemini write a nicer one
Docs	`docs/data_dictionary.md`	Column meanings and conventions; the agent reads this
Tests	`tests/`	Offline tests on a tiny fake dataset (views, SQL guard, loaders, grader, provider loops)

Why DuckDB: it reads CSVs directly, needs no database server, and runs in Colab with one `pip install`.

Why views: the raw Kaggle files have quirks a model would trip over on every question. The views fix them once (see section 3): friendly column names, `NA` text turned into real nulls, traded players de-duplicated, and per-game, totals and advanced stats joined into one table. It is the same reason business teams build a semantic layer.

Safety guards (`db.py`): one statement only, `SELECT` only, external file access disabled after load, configuration locked, 20 second timeout, 50-row result cap.

3. Data

Dataset: Kaggle `sumitrodatta/nba-aba-baa-stats`, scraped from Basketball-Reference. Check the dataset's license and Basketball-Reference's terms before sharing anything beyond this class project.

It is season-level, not game-level. There is one row per player per season (and per team per season), covering 1947 through 2026 including the BAA and ABA. There are no individual box

scores, so questions like "how many 40-point games did he have" are deliberately in the eval as questions the agent must decline.

Raw file	Becomes
Player Per Game, Player Totals, Advanced	`player_season`, `player_season_team` (stats, totals and PER/WS/BPM/VORP in one row)
Team Summaries, Team Stats Per Game, Opponent Stats Per Game	`team_season`, `league_season`
Player Award Shares	`awards` (MVP, ROY, DPOY, MIP, 6MOY, Clutch POY)
All-Star Selections	`all_star`
End of Season Teams	`all_league_teams` (All-NBA, All-Defense, All-Rookie)
Draft Pick History	`draft`
Player Career Info	`players` (height, weight, birth date, college, Hall of Fame)
Team Abbrev	`team_abbrev`

The dataset also has Per 36, Per 100 Possessions, Shooting, Play-By-Play and other files. They are not loaded yet; they are good stretch extensions (add the file to `src/schema.py`, a view to `src/db.py`, and a line to the dictionary).

Full column list: `docs/data_dictionary.md`.

Data traps the views handle (and the agent is told about)

These are the kind of issue that makes an AI agent confidently wrong, so they are worth explaining in your write-up.

Trap	What goes wrong	Fix
`season` is the year the season **ends**	"2023-24" is stored as `2024`; asking for 2023 returns the wrong year	`season_label` column; prompt tells the agent the convention
Traded players have a combined row (`2TM`) **plus** a row per team	Leaderboards list a player twice; career sums double count	`player_season` keeps one row per player per season
Missing values are the text `NA`	Number columns load as text and break averages	Loader reads `NA` as null
Nulls mean "not recorded in that era"	Blocks before 1973-74 treated as zero	Dictionary and prompt say null is not zero
`playoffs` in the team tables means "made the playoffs"	Mistaken for playoff statistics	Renamed `made_playoffs`
A "League Average" pseudo-team row	Shows up as a team in rankings	Split into `league_season`

Trap	What goes wrong	Fix
Rounded values tie	The career points-per-game leader ties at 30.1 (Jordan 30.12 vs Chamberlain 30.07)	Prompt says do not round before sorting

4. Setup (Colab)

1. Open `notebooks/colab_runner.ipynb` in Colab. Its first cell clones this repo.
2. Add `OPENAI_API_KEY` and `GEMINI_API_KEY` under Colab **Secrets** and enable notebook access for each.
3. Run `python src/list_models.py` and set `--model` (or `NBA_OPENAI_MODEL` / `NBA_GEMINI_MODEL`) to models your keys can actually use. The defaults in `src/llm.py` are only starting points.
4. Set the folder with your Kaggle CSVs (`DATA_DIR` in the notebook, or `NBA_DATA_DIR`), then follow the checklist above.

Locally:

```
        pip install -r requirements.txt
        export OPENAI_API_KEY=...
        export GEMINI_API_KEY=...
        export NBA_DATA_DIR=/path/to/kaggle/folder
        python src/list_models.py
        python src/check_data.py
        python -m pytest tests -q
        NBA_PROVIDER=gemini python src/agent.py "Who led the league in assists per game in 2024-25?"
        python src/evaluate.py --provider openai --judge
```

To download the dataset yourself in Colab: `import kagglehub; kagglehub.dataset_download("sumitrodatta/nba-aba-baa-stats")` (needs a Kaggle login in Colab). The loader finds the CSVs in any subfolder.

5. Evaluation

Does running the eval make the model better?

No. The eval only **measures** the agent. The model's weights never change. Accuracy improves when *you* change something because of what the eval showed: the prompt, the data dictionary, the views, worked examples, or the model.

Two traps to avoid:

- **Overfitting to the eval.** If you keep tuning until the same 42 questions pass, the score rises because you fitted those questions, not because the agent is better. So the questions are

split: 28 are `dev` (tune against these, as often as you like) and 14 are `test` (held back). Run `--split test` once at the end. If dev is high and test is much lower, you overfit. Never copy test questions into prompt examples.
- **Noise.** LLM answers vary from run to run, so a one-question change is not a real result. Compare averages over 2–3 runs, and look at the failure types, not just the score.

Also do not run it constantly for its own sake: every run costs API calls. A good loop is: run dev, read the failures, make one change, run dev again.

`eval/questions.csv` has 42 questions in 9 categories: lookup, leaderboard, career, comparison, team, awards, history, advanced, and **unanswerable** (the agent must decline, not invent). Each answerable question has a gold SQL query, and the harness runs it against your data, so the expected answers always come from the same files the agent uses. The gold answers were checked against known facts (Luka Dončić 33.9 ppg in 2023-24, Curry's 402 threes in 2015-16, the 73-9 Warriors, Kareem's six MVPs, and so on).

Two graders:

- **auto**: every value in the gold result must appear in the answer (numbers within rounding).
- **judge** (`--judge`): a model compares the answer with the gold result. More forgiving of phrasing. By default the judge is the *other* provider than the agent, to avoid a model favoring its own family.

Both have blind spots, so read a sample of failures by hand. Failures are recorded in `results/eval_*.csv` with the queries the agent ran, so you can tag each as: wrong column, bad join, missing filter, season off by one, double counted trade, misread question, hallucinated number, or grader error.

Experiments that make a good write-up (change one thing at a time, rerun `--split dev`):

1. Baseline: no data dictionary and no views (let the agent query the raw CSV tables). This shows what the views buy you.
2. Add the dictionary and views (the current design).
3. Add 5–10 worked question-to-SQL examples to the system prompt.
4. Model comparison: OpenAI vs Gemini, and a small vs larger model from each (`--provider`, `--model`); plot accuracy vs cost and latency.
5. Add your own questions (put new ones in `dev` or `test` by setting the `split` column).

Report accuracy overall and per category, and the failure-type breakdown. A statement like "views and dictionary moved accuracy from X% to Y% and removed most season-off-by-one and double-counting errors" is the result worth presenting.

6. Building the app with the models

The plain chat interface is `src/app.py`. For something nicer, the OpenAI and Gemini models write the interface themselves. You give them a brief; they do the design and the code.

- `docs/app_brief.md` is the brief. It says what the agent provides and what the app must do (answer, collapsible SQL, examples, provider choice, loading and error states, mobile friendly). It deliberately leaves all visual design to the model.
- `src/build_app.py` sends the brief to a model, checks the reply is valid Python that uses the agent, flags risky lines, and saves it to `src/app_generated.py`. It never overwrites `src/app.py`.

```
        python src/build_app.py --provider gemini                    # first draft
        python src/build_app.py --provider openai --out src/app_openai.py   # same
brief, other model: compare them
        python src/app_generated.py                                  # launch it (prints
a public link in Colab)
        python src/build_app.py --provider gemini --feedback "make the answer card
larger and use a darker theme"
        python src/build_app.py --provider gemini --feedback "<paste the error you got>"
```

Tips: a stronger model usually writes better code (`--model`, see `list_models.py`); generating one app per provider and comparing them is a nice small experiment; and the feedback loop works for errors too, so paste the traceback back in. **Read the generated file before you run it**: it runs with your API keys in the environment. Edit `docs/app_brief.md` if you want different features.

7. Build plan

Phase	Work	Done when
1. Data	Point at the Kaggle folder, run `check_data.py`, tests and `--check-gold`	Sanity checks pass and gold answers look right
2. Agent v1	Run `agent.py` on 10 questions by hand with each provider	End-to-end answers with queries shown
3. Measure	Full eval for both providers, categorize failures	Baseline accuracy recorded
4. Improve	Experiments in section 5, one change at a time	Before/after table
5. App	Have the models build the interface, compare, iterate with feedback	A shareable chat app
6. Present	Demo plus results and lessons	Short deck or notebook walkthrough

8. Mapping to your business work

Here	At work
The 12 Kaggle CSVs	Fact and dimension tables, or exports

Here	At work
`player_season` / `team_season` views	Governed metric definitions (a semantic layer)
`data_dictionary.md`	Metric glossary, the single most valuable artifact
Min-games filter, `lg = 'NBA'`, end-year seasons	Business rules the agent must apply every time
The data traps table	Real data-quality problems: duplicates, null markers, calendar conventions
Eval set with gold SQL	Regression tests you rerun on every prompt or model change
"Data can't answer that" cases	Trust: an agent that refuses beats one that guesses
SQL guard	Read-only access, row limits, timeouts for any real data source

9. Risks and limitations

- **Wrong-but-plausible answers** are the main risk. Mitigation: views, dictionary, eval, showing the queries.
- **Ambiguity** ("best player", "last season", "career" across leagues) needs a stated assumption. The prompt requires the agent to name its filters, and career questions default to NBA only.
- **Cost and rate limits**: the full eval makes a few hundred API calls. Run `--limit 5` first and check usage in each provider's dashboard. Free tiers return 429 errors under load; `llm.py` backs off and retries.
- **Stateless**: each question starts fresh, so follow-ups like "what about Embiid?" do not work yet.
- **Freshness**: the Kaggle files are a snapshot (this copy runs through the 2025-26 season). The agent states what the data covers.
- **Not tested live yet**: the OpenAI and Gemini loops are tested offline with fakes. The first real run in Colab is the real test, so start with `--limit 5`.

10. Stretch ideas

- Gemma (open model) as a third contender, run on a free Colab GPU, to compare against the API models.
- Load the Per 100, Shooting and Play-By-Play files for questions about shot location, dunks and on/off numbers.
- Multi-turn memory for follow-up questions.
- A chart tool (matplotlib) so the agent can answer "show me ..." questions.
- Retrieval over articles or notes for qualitative context.
- Fine-tune a small open model (LoRA) on synthetic question-to-SQL pairs for this schema and compare it with the API models on your eval.

Appendix A: Data dictionary

Source: Kaggle `sumitrodatta/nba-aba-baa-stats` (Basketball-Reference data). Each CSV is loaded into DuckDB and exposed through the views below. All data is **season-level**: there are no individual game logs.

The most important conventions

- **`season` is the year the season ENDS.** 2023-24 is `season = 2024`; 1961-62 is `1962`. Every view also has `season_label` (`'2023-24'`). Show the label to users, filter on the number. Exception: `draft.draft_year` is the calendar year of the draft.
- **`lg`** is `NBA`, `ABA` (1968-1976) or `BAA` (1947-1949). The NBA starts in `season = 1950`. Filter `lg = 'NBA'` unless the user asks about the ABA/BAA; career totals should normally be NBA only (say so).
- **Player statistics are regular season only.** There are no playoff player stats, no game logs, no salaries, no injuries, no tracking or play-by-play data.
- **Traded players:** in a season a player was traded, the raw data has a combined row (team `2TM`, `3TM`, ...) plus one row per team. `player_season` keeps one row per player per season (the combined row), so it is safe to rank and to sum across seasons. `player_season_team` has every row, for "what did he do with team X" questions. Never add up `player_season_team` rows across a whole season.
- **Names:** different players can share a name, so group by `player_id`. Names carry accents (Jokić, Dončić): use `strip_accents(player) ILIKE '%jokic%'` or the exact spelling.
- **NULL means "not recorded", not zero.** Steals, blocks, offensive rebounds: from 1973-74. Turnovers: from 1977-78. Three-point stats: from 1979-80. Usage rate: from 1977-78. BPM and VORP: from 1973-74. Minutes: from 1951-52.
- **Percentages are fractions** (0.452 = 45.2%), except the advanced "percent" stats (`usg_pct`, `ast_pct`, `reb_pct`, `stl_pct`, `blk_pct`, `tov_pct`), which are already on a 0-100 scale.
- **Team abbreviations follow Basketball-Reference**: BRK (Nets), CHO (Hornets since 2015), PHO (Suns), etc., and historical ones (SEA, NJN, NOH, VAN). Use `team_abbrev` to translate to names.

Views

player_season (one row per player per season) and player_season_team (one row per player per season per team)

Identity: season, season_label, lg, player_id, player, age, team, pos, g (games played), gs (games started). Per game: mpg, ppg, rpg, orpg, drpg, apg, spg, bpg, tpg, pfpg, fgm_pg, fga_pg, fg3m_pg, fg3a_pg, ftm_pg, fta_pg. Shooting: fg_pct, fg3_pct, ft_pct, efg_pct, ts_pct (true shooting). Season totals: total_min, total_pts, total_reb, total_ast, total_stl, total_blk, total_tov, fgm, fga, fg3m, fg3a, ftm, fta, triple_doubles. Advanced: per, usg_pct, ast_pct, reb_pct, stl_pct, blk_pct, tov_pct, ows, dws, ws (win shares), ws_48, obpm, dbpm, bpm, vorp.

team_season (one row per team per season)

season, season_label, lg, team_name, team_abbrev, made_playoffs (TRUE if the team made that year's playoffs; **not** playoff statistics), games, wins, losses, win_pct, pythag_wins, pythag_losses, ppg, opp_ppg (points allowed per game), margin (average point margin), sos, srs (simple rating system), o_rtg, d_rtg, n_rtg (points per 100 possessions), pace, ts_pct, efg_pct, tov_pct, orb_pct, drb_pct, fg3a_rate, ft_rate, avg_age, fg_pct, fg3m_pg, fg3_pct, rpg, apg, spg, bpg, tpg, arena, attendance, attendance_per_game. Rating and pace columns start in the 1950s.

league_season (one row per league per season)

season, season_label, lg, ppg_per_team, pace, off_rating, ts_pct, efg_pct, fg3a_rate, ft_rate, fg3m_pg_per_team, fg_pct, avg_age. Use for comparing eras.

awards

season, season_label, award, player, player_id, age, first_place_votes, pts_won, pts_max, share (vote share), winner (TRUE for the winner). `award` values: `nba mvp`, `nba roy`, `nba dpoy`, `nba mip`, `nba smoy`, `nba clutch_poy`, `aba mvp`, `aba roy`, `baa roy`. (Finals MVP is not in this data.)

all_star

season, season_label, lg, player, player_id, all_star_team, replaced (TRUE if an injury replacement).

all_league_teams

season, season_label, lg, team_type (`All-NBA`, `All-Defense`, `All-Rookie`, `All-ABA`, `All-BAA`), team_number (`1st`, `2nd`, `3rd`), player, player_id, position.

draft

draft_year, lg, overall_pick, round, team, player, player_id, college (NULL for players with no US college).

players (one row per player, career info)

player_id, player, pos, height_in, weight_lb, birth_date, colleges, first_season, last_season (end-year convention), debut, hall_of_famer.

team_abbrev

season, lg, team_name, abbreviation, made_playoffs.

Query tips

- Leaderboards need a minimum-games filter (`g >= 50` is a reasonable default for per-game stats) or a minimum minutes filter for advanced stats (`total_min >= 1000`); otherwise a player with 2 games tops every list.

- Do not round before sorting: ties after rounding hide the real leader. Round only the displayed value.
- Career total = `sum(total_pts)` over `player_season` for one `player_id`. Career per-game average = total divided by `sum(g)`, not the average of the yearly averages.
- "Last season" means the most recent season in the data (`max(season)`), not the current calendar year.

Appendix B: Evaluation questions

#	Category	Split	Question
1	lookup	dev	How many points per game did Luka Dončić average in the 2023-24 season?
2	lookup	dev	How many games did LeBron James play in the 2017-18 season?
3	lookup	test	What was Nikola Jokić's true shooting percentage in 2023-24?
4	lookup	dev	How many three-pointers did Stephen Curry make in the 2015-16 season?
5	lookup	dev	How many points per game did Wilt Chamberlain average in 1961-62?
6	lookup	test	What was James Harden's scoring average in 2021-22, counting his whole season across both teams?
7	leaderboard	dev	Who led the NBA in points per game in 2023-24 (minimum 50 games)?
8	leaderboard	dev	Who were the top 3 in assists per game in 2024-25 (minimum 50 games)?
9	leaderboard	test	Who made the most three-pointers in a single season in NBA history?
10	leaderboard	dev	Who had the highest PER in 2022-23 among players with at least 1,000 minutes?
11	leaderboard	dev	Who led the NBA in blocks per game in 2023-24 (minimum 50 games)?
12	leaderboard	test	Who had the most triple-doubles in a single NBA season?
13	career	dev	How many career points did LeBron James score through the 2024-25 season?
14	career	dev	Who has the most career three-pointers made in the NBA?
15	career	test	How many NBA seasons did Vince Carter play?
16	career	dev	Who has the highest career points per game among players with at least 10,000 career points?
17	career	dev	Who has more career NBA points, Kobe Bryant or Dirk Nowitzki?
18	comparison	test	Compare Michael Jordan and LeBron James by career points, rebounds and assists per game.
19	comparison	dev	Who averaged more points per game in 2022-23, Joel Embiid or Nikola Jokić?
20	comparison	dev	How did Stephen Curry's points per game change in each season from 2014-15 through 2017-18?
21	team	test	Which team had the best record in 2015-16, and what was it?
22	team	dev	Which team has the highest Simple Rating System (SRS) in a single season in NBA history?
23	team	dev	Which team allowed the fewest points per game in 2023-24?
24	team	test	How many wins did the Oklahoma City Thunder have in 2024-25?
25	team	dev	Which team played at the fastest pace in 2023-24?

#	Category	Split	Question
26	awards	dev	Who won the NBA MVP in 2023-24?
27	awards	test	How many MVP awards has LeBron James won?
28	awards	dev	Which player has won the most NBA MVP awards?
29	awards	dev	Who won Rookie of the Year in 2023-24?
30	awards	test	How many times was Tim Duncan named to the All-NBA First Team?
31	awards	dev	How many times was Kobe Bryant an All-Star?
32	history	dev	Who was the number one overall pick in the 2003 NBA draft?
33	history	test	Which college has produced the most number one overall NBA draft picks?
34	history	dev	Which season had the highest league-wide pace, and what was it?
35	history	dev	What was the league-average points per team per game in 1983-84 compared with 2023-24?
36	advanced	test	Who had the highest usage rate in a single season (minimum 1,500 minutes)?
37	advanced	dev	Who had the best box plus/minus (BPM) in 2023-24 among players with at least 1,500 minutes?
38	advanced	dev	Which player had the most win shares in a single season in NBA history?
39	unanswerable	test	What is LeBron James' salary this season?
40	unanswerable	dev	How many points did Michael Jordan score in the 1993 NBA Finals?
41	unanswerable	dev	How many 40-point games did Devin Booker have in 2023-24?
42	unanswerable	test	Who will win the 2027 MVP award?

Appendix C: App brief given to the models. Build the chat interface for the NBA History Analyst

You are writing the user interface for a basketball question-answering agent. The agent already works. Your job is the interface only. Make it polished, distinctive and pleasant to use. The visual design is entirely your call: layout, theme, colors, typography, header, icons, spacing, and any small touches that make it feel finished. A basketball-inspired look is welcome, but readability comes first.

What the agent gives you (do not change it)

```
        from agent import NBAAgent            # the file is run as `python
src/<yourfile>.py`, so this import works

        agent = NBAAgent(provider="openai", model=None)   # provider is "openai" or
"gemini"; model=None uses the default
        result = agent.ask("Who led the NBA in scoring in 2023-24?")

        result.answer    # str: the plain-English answer (may contain markdown)
        result.queries   # list[str]: every SQL query the agent ran, in order
        result.errors    # int: how many of those queries failed (the agent retries, so
this is informational)

        agent.db.coverage()   # str: one or two sentences describing the data's seasons
and limits
```

- `ask()` takes roughly 5 to 60 seconds and is stateless (each question is independent, so there is no chat memory).
- `ask()` can raise exceptions (bad API key, rate limit, network). Catch them and show a short, friendly message.
- The data is season-level (one row per player per season), regular season only. There are no game logs, playoff player stats, salaries or injuries. The agent will say so when asked about those.
- API keys are already in the environment (`OPENAI_API_KEY`, `GEMINI_API_KEY`). Never ask for, display or log a key.

Must have

1. A question box and a button (Enter should also submit).
2. A provider choice (OpenAI or Gemini). Create each `NBAAgent` once and reuse it; do not rebuild it per question.
3. A clear loading state while the agent works, and a friendly error state if it fails.

4. The answer shown prominently and readably.
5. A collapsible "How I got this" section that shows the SQL queries, formatted as code. Hide it when there are none.
6. Clickable example questions, using these:
    - Who led the NBA in points per game in 2023-24 (minimum 50 games)?
    - Compare Michael Jordan and LeBron James by career points, rebounds and assists per game.
    - Which team had the best record in 2015-16?
    - Who has the most career three-pointers made in the NBA?
    - Who won Rookie of the Year in 2023-24?
    - How many points did Michael Jordan score in the 1993 NBA Finals?
7. A short note showing `agent.db.coverage()` so users know what the data covers.
8. A history of the questions asked in this session, so users can scroll back (the agent itself has no memory).
9. It must work on a phone-sized screen too.

Technical constraints

- One Python file, using Gradio (already installed; code for the installed version is below). Use only APIs that exist in that version.
- Launch with `share=True` so it works in Google Colab, and put the launch call under `if __name__ == "__main__":`.
- No other third-party packages. No `eval`, `exec`, `subprocess`, `os.system`, file writes, or network calls of your own. Everything goes through `NBAAgent`.
- Add a short comment at the top saying what the file is. Keep the code readable: someone new to Gradio should be able to follow it.

Output format

Reply with exactly one fenced ```python code block containing the complete file, and nothing else.